

QSpace Documentation QSpace: MEX and QSpace member functions in Matlab

QSpace (2006-2024) by A. Weichselbaum

This documentation was auto-generated by QSpace.m $\xrightarrow{\text{tex}}$ QSpace.pdf
based on the current help and usage information.

Current version: QSpace v4.0
Generated: February 22, 2024

Contents

1	MEX routines (./bin folder, core routines)	2
	compactQS	2
	contractQS	2
	diagQS	7
	eigQS	7
	getDimQS	8
	getIdentityQS	8
	getQDimQS	10
	getRC	11
	getSymStates	11
	isHConjQS	12
	isIdentityCG	12
	isIdentityQS	12
	makeUniqueQS	13
	normQS	13
	orthoQS	13
	permuteQS	15
	plusQS	15
	setupRCStore	16
	skipZerosQS	16
	svdQS	17
	traceQS	18

2	MEX routines (./bin folder, other)	19
	getSmoothSpec	19
	getSmoothTDM	19
	fdmNRG_QS	21
	fgrNRG	24
	NRGWilsonQS	27
	tdmNRG	31
3	MEX routines (./util folder, selected)	34
	matchIndex	34
	mpfr2dec	34
	wbrat	35
	helloworld	35
	mexworld	35
4	@QSpace class member functions (Matlab)	36
	acomm	36
	addSymmetry	36
	appendSingletons	36
	cat	37
	checkOM	37
	checkSameSpace	38
	chopd	38
	comm	38
	conj	39
	contract	39
	ctranspose	39
	datasize	39
	decomposeOp	39
	deconj	40
	diag	40
	dim	40
	display	40
	eig	41
	eq	41
	finditag	41
	fixAbelian	42
	fixScalarOp	42
	get_qdim	42
	getAtensorLoc	42
	getEQdata	43
	getId	43
	getIdentity	43
	getIdentity3	44
	getitags	44
	getproj	44

getqdim	45
getqdir	45
getqfmt	45
getQRange	45
getQSpectra	45
getscalar	46
getsub	46
getsym	47
getvac	47
getzdim	48
gotCGS	48
imag	48
info	48
info3	49
isAbelian	49
isdiag	49
isempty	50
isequal	50
isHConj	50
isop	50
isOpConj	51
isOrtho	51
isreal	51
isscalar	51
isscalarop	52
issingleop	52
issingleton	52
itag2iter	52
itagrep	53
itags2int	53
itags2odir	53
itags2str	53
logical	54
makeIrop	54
mapQ	54
markitags	54
minus	55
mrdivide	55
mtimes	55
norm	56
norm2	56
not	56
numsym	56
olap	56
oplus	57
ortho	57

permute	57
plotEdata	58
plotQSpectra	58
plus	58
QSpace	59
randomize	59
rank	60
real	60
rmitags	60
rmtrace	60
sameas	61
SEntropy	61
SetCC	61
setitags	62
SetOL	63
showdeg	63
skipprec	64
skipzeros	64
sort	64
split_ops	64
sqrt	65
squeeze	65
subsasgn	65
subsref	66
sum	66
symperm	66
symrank	66
times	66
trace	67
transpose	67
uminus	67
unmark	67
untag	68
uplus	68

1 MEX routines (./bin folder, core routines)

>> compactQS

Usage: S=compactQS([opts],'qtype',Q1,Q2,Q,A [,perm]);

reduce matrix elements of the rank-3 IROP.

Here A is still specified in full (dense) tensor format with matrix elements given by

$$A(i1,i2,io) := \langle i1|A(io)|i2\rangle$$

NB! this assumes the index (1,2,op) specified on the lhs! The symmetry types are specified by qtype. Moreover, all states i1 and i2 must be already cast into symmetry eigenstate grouped into multiplets (see getSymStates.cc) with q- and z-labels both present and interleaved in each input state label sets Q*.

The state spaces are combined in the order (Q2+Q =>Q1), thus using the Clebsch Gordan coefficients (Q1,Q|Q2). This is a natural order in that, Q=-1 annihilates a particle on a |ket> state (the annihilation operator on a ket state increases charge, i.e. Q would be +1, in case Q1 and Q had been combined into Q!). This therefore represents a natural index order.

Options

perm permutation to be applied to {Q1,Q2,Q} and A prior to combining first two indices into third.

'-h' display this usage and exit

(C) AW : Apr 2010 ; Jul 2011 ; Oct 2014

>> contractQS

function contractQS()

Contract set of QSpace tensors in a pairwise fashion as described with usage #1 and #2 below. This automatically also takes care of the underlying Clebsch Gordan coefficient spaces if present based on X-symbols.

Each QSpace can be used as is, or as its 'conjugate' where the

conjugate of a QSpace A, i.e. `conj(A)` is defined as the QSpace

- 1) with all ``arrows'` reversed
- 2) keeping the SAME qlabels on all legs
- 3) and complex conjugation of all data{} if applicable,
i.e., if the (reduced) matrix elements are complex.

Because of (1), the specification of conjugate flags (``conj-flags'`) is also important for QSpaces with all-real matrix elements. For reversing individual arrows, see `getIdentitQS(.., '-0')`.

Usage #1: `S=contractQS(A, ica, B, icb [, perm, OPTS ])`;

Plain contraction of a single pair of QSpaces, A and B, with respect to given explicitly specified sets of contraction indices ica and icb, which can be specified

- numerically (e.g. `[1 2]`),
- or as strings (e.g. `'1,2'`, or `'1 2'`)
- or as compact strings (e.g. `'1,2'`, or `'12'`)

The last 'compact format' is only possible / unique, of course, if the tensors A and B do not have more than 9 legs (which basically never occurs), such that the contraction indices remain in the single digits (this can be further relaxed, though, by extending the digital range to letter, i.e. using a-z after 0-9).

The recommended way to specify conj-flag with usage #1 is together with the contraction indices in string notation! For example,

```
contractQS(A,[1 3],B,[1 3],'conjA')  is equivalent to
contractQS(A,'1,3;*',B,[1 3])         is equivalent to
contractQS(A,'13*',B,'13')
```

(deprecated) options specific to usage #1:

```
'conjA'  use complex conjugate of A for contraction
'conjB'  use complex conjugate of B for contraction
```

Usage #2: `S=contractQS(A,...,{B,...,{C,...}},... [, perm, OPTS ])`;

Generalized 'cell-contraction' of tensors: when encountering a cell, the content of that cell is contracted first, before using its result. This allows the specification of an entire

patter of pairwise contractions based on a nested cell structure where the lowest-level contractions are performed first. An optional fully enveloping outer cell bracket at 'base' level as in `S=contractQS({A,...,{B,...,{C,...}},...} [, perm, OPTS ])` is permitted, which may be used e.g., when debugging nested parts of cell-contractions.

Cell contractions are furthermore based on QSpace 'itags' i.e. string labels for indices with up to 8chars, and which are specified in `QSpace.info.itags`. This offers automated contraction ('auto-contraction') of pairs of tensors solely based on matching itags! Usage #2 therefore does not (also) allow explicit specification of contraction indices as in usage #1. Therefore itags (plus conj-flags) must be unique.

itags must always also contain individual conjugate flags (this represent the bare minimum that must be specified with each QSpacein v3): the conjugate flags on individual indices (legs) of a tensor determine whether that index (leg) is in- or out-going, with the convention that

all out-going indices have a trailing * in their itags!

For example, an A-tensor with L(ef), R(ight), s(=local) indices may have `itags A.info.itags={'L','R*','s'}` assuming (L,R,s) index order.

In usage #2, for every operator additional optional strings can be specified, appearing right after the affected tensor e.g. QSpace A:

A,'*' apply overall (complex) conjugation on given input tensor A (see early comments above)

A,'!ij' do not contract indices specified by ij (in compact format) despite they share common matching itags.

A,'!ij*' both of the above in a single instruction
 the conjugate flag '*' always trailing.

A,'ij' [07/2023]

A,'ij*' explicitly specify indices to contract
 this is intended only for the case of degenerate itags
 i.e., when identical itags appear on multiple legs
 including the same conjugate flag.

itags may be set or adapted on the fly (this is performed prior to the auto-contraction together with the above directions)

X, '--op:<tag>[:<opl=op>]'

specify itags for given (e.g. local) operator.

The last option considers X an operator, and hence assumes operator itags '<tag>;<tag>*[:opl*]'

for QSpace X; the default operator label is 'op', but may be changed by specifying a trailing ':<opl>' as indicated above. As a safeguard, this issues a warning, if existing itags are overwritten. This is relevant e.g. for local operators that are applied to a very specific site with associated itag.

[11/2018] the specified <tag> may now also represent a regular expression (regex), recognized by non-alphabetic special characters, while ignoring conj-flags (without special characters, the specified <tag> is taken as is!). This usage then searches for a matching itag in the paired up QSpace (cell) in the contraction.

Ex. Consider A some QSpace with a single local index that starts with 's', e.g., like 's010' for site 10; then contract(A,Xloc,'-op:~s') will autocontract the local operator Xloc to the correct local index in A (here the regex '~s' indicates 'starts with s'); An alternative operator itag may still be specified by adding a trailing ':opl' as indicated earlier.

A, '--itag:s/pat/rep/[gi] [07/2023]

replace/modify existing itags on the fly for a particular recursive level of a cell contraction based on regular expressions (regex; using ECMAScript grammar, cf. C++/regex). The syntax is much analogous to perl regex.

The trailing flags enable case insensitive replacement [i] and global replacement [g] of all possible matches (by default, only the first match is replaced).

An adaptation of usage #2 can also be used for plain sequential contractions

S=contractQS(A [,flagsA],B [,flagsB],C,... [, perm, OPTS])

which is equivalent to

S=contractQS({A [,flagsA]},{B [,flagsB], {C,...}}}, [,perm,OPTS]).

That is, by grouping $A \cdot (B \cdot (C \cdot \dots))$, sequential contractions start

from the right end onwards to the beginning of the specified set. This structure is also permitted at any lower level inside cell contractions. Non-contracted indices are always collected in the order they appear in the input.

The remaining trailing OPTS are

`perm` optional permutation (non-string) on the final object; this permutation can be shorter than the actual rank of the resulting QSpace, which then is completed as an identity permutation on the remainder of indices.

By permitting the specification of indices to contract also in contractions based on itags (cell-contractions) there is a potential ambiguity of whether `perm` should be interpreted as indices on the last specified tensor (the latter may also be represented in compact string format for better differentiation).

Hence the rules for `perm` to be interpreted as permutation requires that (i) it is a valid permutation of length $r \geq 2$ to start with, and (ii) that it is in numeric format, i.e., not written as compact string. Valid examples are `[2 1]`, `[2 3 1]`, but not, e.g., `1`, `[2 3]`, or `'21'`.

The ambiguity above is fully circumvented when also wrapping the last contraction at base level into a cell, such as `contractQS({A,...},perm [, '-v'])` [see above], in which case `perm` is always interpreted as permutation on the overall result, irrespective of rank or whether specified as numeric or compact string index. In case of a string, this then also permits to specify an optional conjugate flag to be applied on the overall result.

`'-v'` debug mode that shows all levels of a cell contraction together with the actual contractions performed based on shown itags. Internally, degenerate itags are frequently flagged by making use of signed characters, which makes them unique and thus differentiate them. When printed, flagged itags are formatted as `"<itag>|b"` where the character ``|'` separates the actual itag string from number `b` (int8 decimal) which in its binary form reflects the internal flags.

Mixed usage of #2 and #1 is not possible.

Usage #2 is typically recommended because auto-contraction

makes it far easier to contract entire networks in a single go without having to manually track and specify index locations.

AW (C) May 2010-2023

>> diagQS

Usage: `dd=diagQS(A)`

Get diagonal data of given QSpace (for rank-2 QSpaces only).

Wb (C) Aug 2016

>> eigQS

Usage: `[E [,I]]= eigQS(H [,opts])`

Obtain eigenspectrum and eigenbasis of input 'Hamiltonian' H in QSpace representation. The eigendecomposition is split into kept (K) and discarded (D) state space, as governed by the truncation parameters Nkeep and Etrunc (for actual Hamiltonians) or Rtrunc (for density matrices $R:=H$).

Options

'Nkeep',... max. number of states to keep (default: -1 = keep all)
'Etrunc',... keep states according to energy truncation criteria with $E \leq E_0 + E_{\text{trunc}}$ (default: ≤ 0 = ignore; note that Etrunc is interpreted relative to lowest eigenvalue $E_0 = \min[\text{eig}(H)]$).
'Rtrunc',... keep states according to truncation criteria of reduced density matrix with $\text{eig}(R) > R_{\text{trunc}}$ with $R:=H$ (this is alternative to Etrunc and thus ignored by default; note, furthermore, that here Rtrunc is interpreted absolute, i.e. as is).
'deps',... allow to keep (close to) degenerate spaces together (default: 0, i.e. sharp truncation e.g. with Nkeep)

Output

E contains the full sorted eigenspectrum (1st column), and in the presence of non-abelian symmetries, the overall multiplet degeneracy as a 2nd column.

The output/info structure I contains the fields

.AK, .AD for kept and discarded state space
 .EK, .ED, for kept and discarded eigen spectrum (diag)
 .DB block dimension of input Hamiltonian H in terms
 of multiplets (1st col) and states (last col).
 .NK actual number of kept multiplets / states.
 .Etrunc or
 .Rtrunc actual truncation threshold used

AWb (C) Sep 2006 ; Mar 2013 ; Aug 2015

>> **getDimQS**

Usage: [D[,D2]] = getDimQS(A)

get full dimension of given QSpace object.

In case of non-abelian symmetries, usage #2 returns
 number of multiplets as D, and the full state space
 dimension as 2nd argument D2.

NB! if no second output argument (D2) is requested,
 this routine returns D if D==D2 (e.g. which is always
 the case of abelian symmetries) but [D;D2] if D!=D2.

(C) AW : May 2006 ; Oct 2014

>> **getIdentityQS**

Usage 1: E=getIdentityQS(A [,i1, perm,'-[0z]']);

get plain identity operator from given QSpace.
 corresponding to the state space on index i1 with A.

Options

'-0' this generates the 1J symbol, i.e. an identity operator
 with all indices inward. This can be used to "revert arrows",
 with the effect that the second index on E carries the
 dual state space with respect to the first index.

For this reason, the respective itag, as inherited from A @ i1,
 if non-empty, is also changed by flagging it with a trailing
 prime ('). Therefore in order to accommodate for this additional
 character, the itag in A @ i1 must have less than 8 chars in length.

This flag is considered part of the itag, hence overall such a flagged itag is considered different from the original. The flagging also acts like a toggle, i.e., marking an itag that is flagged already, removes the trailing prime.

For readability, the QSpace display shows such 'marked' itags in color gray while hiding the trailing prime. This behavior can be turned off by setting the environmental variable `QS_LOG_COLOR` to zero (default: 1).

'-z' same as '-0' but without flagging the itag in E @ index 2.

Usage 2: `A=getIdentityQS(A [,i1 [,ta]], B [,i2 [,tb]], t3, perm)];`

get tensor product space of input spaces defined by A and B. If `i[12]` is not specified, rank-2 objects are assumed taking (the first) two indices.

Default index order of the output space is (1,2,12*) with the itags of A and B inherited if present. An itag `t3` for the combined output space (dim3) may be specified. By default, a conj flag if present in `t3` will be ignored, with the following exception:

[06/01/2019] In addition, for an input A-tensor with LRs index order convention, its itags may be inherited to C by specifying `<ta>` xor `<tb>` (see usage above) in the format `'-A[:...]'` where ``..'` indicates extra characters to be concatenated at the end of the itag for the \*un\*fused [LR] index in A or B for `<ta>` xor `<tb>`, respectively.

For example, consider an A-tensor with itags {K01,K02*,s02}, and E a local identity operator without itags; then in order to ensure a complete local state space, one may use

```
>> getIdentityQS(A,1,'-A:~',E)
```

which generates a QSpace with itags {K01,s02,K02~*}.

K01 is inherited from A via ``A@1'`; the remaining two itags are derived from the remaining two itags in A;

therefore ``A,i'` must have `i=1` or `2` with in LRs index order. conj-flags are properly adjusted as needed.

Conversely, `getIdentityQS(A,2,'-A:~',E)`

generates itags {K02,s02,K01~*}, e.g. used for R->L sweep.

[08/2023] The itag `t3` on the fused index may be written in the flagged format `'...'` -> `'-m:...'` or `'-m!...'` which interprets trailing marker (') and conj (*) flags

in the sense described for '-m:..' here (the behavior is simply reversed w.r.t. to the conjugate flag for '-m!..'): When inserting an identity the fused space may already refer to a particular existing itag for that leg. From the perspective of the rank-3 identity A-tensor generated here, if t3 has a trailing conj marker, it already reflects an outgoing index, and hence can be kept. However, if t3 does not have a trailing conj flag, then the identity generated here effectively reverts the arrow. As such it rather refers to the dual space. In order to reflect this in the fused space, the itag t3 will 'marked' as dual state space which by QSpace convention is indicated by a trailing prime (') as part of the itag, hence before the conj flag. This acts as a toggle, i.e., if a trailing prime is already present in t3, adding another one rather removes it, instead.

Finally, the output index order may be changed by specifying a permutation perm to be applied on the final object. If itag is not specified, perm must be in numerical format, otherwise a (compact) string format is also accepted.

Further options

```
'-h'  display this usage and exit
'-v'  verbose
```

Note that mex files do not allow to return class objects, hence A is returned as a QSpace structure. To get a class object, use QSpace(getIdentityQS(...)), or equivalently, if getIdentity() is properly defined as a wrapper routine within matlab's Class/@QSpace (see MPS Pack), having a QSpace input A, this may be shortened to getIdentity(A,...).

(C) AW : Apr 2010 ; Oct 2014 ; May 2017

>> getQDimQS

Usage: [q,dd,dc] = getQDimQS(A,dim)

```
get dimension for every Q in QIDX for given QSpace object
dim is dimension to check (dim=='op' can be used to
consider both dimensions of an operator simultaneously)
```

Data returned

q symmetry labels present (unique; n x nsymlabels)
dd corresponding dimension of reduced data (n)
dc corresponding dimension of Clebsch-Gordan (n x nsym)

AW (C) Jun 2007 ; Oct 2010

>> **getRC**

Usage: C=getRC(cgr [,opt])

get RCStore data as specified by input

Options:

- f returns CGC data in double format (by default just CG reference)
- F returns CGC data in multiprecision format

Usage: C=getRC(<sym>,'--info');

return currently loaded RC data
specifically also <sym>.

Wb,Oct 2015 ; Nov 2016

>> **getSymStates**

Usage #1: [U,I]=getSymStates(Sp,Sz);

get symmetry eigenstates for symmetry operations given by
raising operators Sp and (diagonal) z-operators Sz.
The order of the Sz operators determines the order of the
z-labels to be determined.

Usage #2: [U,I]=getSymStates(SOP)

Moreover, Sp and Sz can be grouped according to each symmetry
by using a single vector structure Is with the mandatory
fields .Sp and .Sz. In this case, an empty Sp is interpreted
as Abelian quantum number. Eventually Sp and Sz are combined
into a single sequence as in usage #1.

Further optional fields

- .type symmetry type (in QSpace convention)
- .qfac applied to all z- and q-labels (scalar or vector)
- .jmap subsequently to qfac, applies linear transform
 of q-labels (square matrix)

Output: symmetry eigenstates written as full matrix in U,
and info structure I with the fields

basically reiterating the input

- .R0 input .Sp and .Sz data
- .type symmetry type
- .JMap J-transformation used (identity if not specified)
- .dz number of z-operators for each symmetry

actual output data

- .dd dimension of irreducible multiplets generated
- .RR irreducible representations with qfac and jmap applied
- .QZ interleaved q- and z-labels.
- .d2 dimensions of setup in QZ (non-abelian number have
the same number appearing twice)

(C) Wb,Jul05,10

>> **isHConjQS**

Usage: [i,istr]=isHConjQS(A [,eps]);

check whether QSpace is hermitian within threshold eps (1E-12).
A must be even-rank object as hyperindex is allowed.

Wb,Aug17,06

>> **isIdentityCG**

Usage: i = isIdentityCG(A)

check whether QSpace A has identity Clebsch-Gordan spaces.
Note that for full abelian spaces, this is always true.

Wb,Jan12,12

>> **isIdentityQS**

Usage: i = isIdentityQS(A [,eps])

check whether QSpace A represents a block-diagonal
identity matrix. This also checks the corresponding
Clebsch-Gordon spaces if any (default: eps=1E-12).

(C) AW : May 2006 ; Oct 2010 ; Oct 2014

>> makeUniqueQS

Usage: A = makeUniqueQS(A [,opts])

MEX wrapper routine for QSpace::makeUnique()

Options

-[qQ] quite mode
-[vV] verbose mode (wrt. outer multiplicity)

AWb (C) Nov 2007 ; 2012

>> normQS

Usage: nrm = normQS(A)

calculates the (Frobenius) norm of given QSpace,
i.e. $|A|^2 = \text{norm}(A)^2 = \text{trace}(A'*A)$, and appropriately
generalized to arbitrary rank. In the presence of CGCs, this
implies $|A|^2 = \sum_i (|\text{data}\{i\}|^2 * \text{prod}_j (|\text{cgs}(i,j)|^2))$.

(C) AW : Jun 2010 ; Oct 2014

>> orthoQS

Usage #1:

[A1,A2,[,info]] = orthoQS(PSI,idx, [,OPTS]);

Orthonormalize QSpace PSI w.r.t. given SINGLE index idx,
which is split off into tensor A1, connected via a trailing
new intermediate index with A2, i.e. $\text{PSI} = \text{A1} * \text{A2}$, where
A2 keeps its index order (i.e. index idx stays in place).

Note that the new intermediate index will typically be
truncated w.r.t. to singular values (see stol below).

Convention on orthonormalization direction (cf. lrdi below,
which is NOT specified here): A1 is ALWAYS an isometry/unitary
that has one in- and one out-index. This allows the new
intermediate index to carry the same itag as idx in PSI.

Usage #2 (abelian symmetries only)

```
[A1,A2[,info]] = orthoQS(PSI,idx, lrdir [,OPTS]);
```

Similar to usage #1 above, except that for abelian symmetries, idx may contain any number of indices consistent with PSI (in contrast, for non-abelian symmetries, idx can only specify a single index (or equivalently r-1 indices with r the rank of PSI).

Usage #3 (abelian symmetries only)

```
[A1,A2[,info]] = orthoQS(A1,A2,ic1,ic2, lrdir [,OPTS]);
```

Orthonormalize QSpace A1 towards index ic1 using SVD, where the resulting tensor, split off from A1, is right away merged with A2 at index ic2 (having lrdir='>>'; vice versa for lrdir='<<').

This internally connects to usage #2, in that the input tensors A1 and A2 are right away contracted; e.g. for lrdir='>>':
PSI = contractQS(A1,ic1,A2,ic2);

INPUT

PSI or PSI=A1*A2, where
A[12] are two nearest neighbor tensor
ic[12] that share the common index specified by ic1 and ic2.

lrdir direction of orthonormalization; this accepts the following equivalent values:
'>>', 'LR', or +1 for "left-to-right"
'<<', 'RL', or -1 for "right-to-left"

OPTIONS

'Nkeep',... max. number of states/multiplets to keep (-1: all)
'Nkmin',... min. number of states/multiplets to keep on index connecting A1 to A2 (default: 0, i.e. no lower bound)
'stol',... absolute tolerance on SVD i.e. on sqrt(rho) default: 1E-8 if Nkeep is not set, 0 otherwise.
'itag',... itag on newly generated index
'-v' print info

OUTPUT

A1[2] orthonormalized A1[2]
info info structure containing (amongst other fields)

.svd vectorized sorted complete singular value data;
 multiplicity (i.e. multiplet dimension) is specified
 in svd(:,2) in the presence of non-abelian symmetries.
 .svd2tr truncated weight (singular values squared).
 .sfac factor to reestablish the original norm(Psi).
 .flag !=0 indicating potential trouble (see .info field).
 .DD (multiplet) dimension of original Psi.
 .D1 (multiplet) dimension of A1.

See also svdQS.

AW (C) Jun 2006; Jun 2009; Aug 2015

>> permuteQS

Usage: A = permuteQS(A, P [, 'conj'])

permute input QSpace using given permutation P.

Optional trailing 'conj' also applies (complex) conjugation
 (note that this also affects real QSpaces in that qdir and
 itags are altered!).

For convenience, P [, 'conj'] may also be represented as
 single string, e.g. [2 1], 'conj' is equivalent to '2,1;*' or
 '21*' where the convention on string notation follows that of
 contraction indices [ctrIdx].

NB! [06/02/2019] the provided permutation can be shorter
 than the rank of the QSpace; in this case it only affects the
 leading range of indices, i.e., acts like an identity on the
 remainder of indices.

AW (C) Aug 2006 ; May 2010 ; Oct 2014

>> plusQS

Usage: C = plusQS(A, B [, bfac, opts])

adds one QSpace to another as in C = A+bfac*B ([bfac=1])
 block-diagonal identity matrix.

Options

'-v' verbose mode (e.g. on remaining outer multiplicity)

(C) AW : Jun 2006 ; Oct 2014

>> setupRCStore

Usage #1: setupRCStore(sym [,opts])

Setup CGC container for given symmetry,
with the data stored in directory specified by the
environmental variable \$RC_STORE.

Usage #2: setupRCStore(sym,q1,q2 [,opts])

explicitly request to generate tensor product
decomposition $q1 \times q2$

Options

'npass',... number of passes through gCG data (3).
'dmax',... specify max. average multiplet dimension $\sqrt{d1 \times d2}$
in tensor product decomposition ($10 \times qlen$).
'-a' generate all, yet compatible with dmax (by default,
multiplets with 2-digit qlabels are no longer used
in decomposition)

Wb (C) Aug 2014 ; Apr 2015

>> skipZerosQS

Usage: [A,n] = skipZerosQS(A [,eps,opts]);

skip data blocks that have all-zero entries to within eps
(default $\text{eps}=1\text{E-}14$). The total number of skipped blocks
is returned as 2nd argument.

For scalar (i.e. rank-2 block-diagonal) operators zero-
blocks are kept, by default (e.g. relevant for Hamiltonians
where states with energy zero are relevant states.
This can be disabled by using the option '--all'.

Wb, May14, 10

>> svdQS

Usage #1:

```
[U,S,Vd,[,info]] = svdQS(PSI,idx [,OPTS]);
```

Compute SVD decomposition of QSpace object PSI, where the indices idx are split off into Vd, and the remaining indices into U, which are '>>' and '<<' orthonormalized, respectively (i.e. are unitaries or isometries generated from the SVD. Typically, the indices in PSI are 'all-in'.

Note that the 3rd argument returns Vd:=V' and not V, i.e. already includes the dagger such that $PSI = U \cdot S \cdot Vd$ e.g., similar to the LAPACK convention, cf. zgesdd.

By convention, if a single index is split off, U maintains the index order of PSI. For rank $r=2$ then, the case $idx=2$ behaves the expected way. However, for $idx=1$ the transpose $U^t = U^\dagger$ is returned for U, having $PSI = [U^t \cdot S \cdot Vd]^t = \text{contract}(S \cdot Vd, 1, U, 1)$.

For non-abelian symmetries, idx is required to be a single index (or conversely, all indices but one). If PSI is grouped into two sets of indices each of which has more than one index, then indices need to be fused prior to calling svdQS. Also, in this case of a single index specified, U always resembles PSI, i.e. has the same rank and index order.

Usage #2 (deprecated):

```
[U,S,V',[,info]] = svdQS(A1, A2, ic1, ic2 [,OPTS]);
```

Compute SVD decomposition of QSpace object $PSI=A1 \cdot A2$, where '*' stands for the contraction $(A1, ic1, A2, ic2)$. The split occurs between the remaining indices in A1 and the remaining indices in A2. These indices are typically 'all-in', while the index/indices ic1 to be contracted with ic2 can have any initial orientation. There will always be a single new intermediate connecting index between A1 and A2.

Input

PSI	or $PSI=A1 \cdot A2$, where
A[12]	are two nearest neighbor tensor
ic[12]	that share the common index specified by ic1 and ic2.

OPTIONS

'Nkeep',... max. number of states/multiplets to keep (-1: all)
'Nkmin',... min. number of states/multiplets to keep on index
 connecting A1 to A2 (default: 0, i.e. no lower bound)
'stol',... absolute tolerance on SVD i.e. on $\sqrt{\rho}$
 default: 1E-8 if Nkeep is not set, 0 otherwise.
'itag',... itag on newly generated index
'disp' display info to stdout

OUTPUT

A1(2) updated and properly orthonormalized A1(2)
info info structure containing
 .SVD singular value data blocked within QS symmetry space
 .svd vectorized singular value data

See also orthoQS.

AW (C) Sep 2008 ; Aug 2015

>> **traceQS**

Usage: x = traceQS(A [,i1,i2])

trace out index set i1 with i2 of given QSpace object;
for even-rank A, missing index sets is interpreted as
full trace of indices i1=1:r/2 with iw=r/2+1:r.
if all indices of A are traced out, the returned x is
a number (QSpace otherwise).

AW (C) 2006 ; 2018

2 MEX routines (./bin folder, other)

>> getSmoothSpec

Usage: [omega, As [,Info]] = getSmoothSpec(om, Araw, 'eps',... [, opts])

om omega (bins) for Araw
Araw raw discrete spectral data; it may contain several set
 in separate columns, e.g. like data for different spins

Options / Flags

'om',... new discretized set of omega values (also returned as omega)
'sigma',... sigma used in log-Gauss broadening (0.6)
'eps',... smoothly replaces log-Gauss with regular Gaussian;
 in absolute units since this routine does not know
 anything about temperature(!). For the same reason,
 eps must be specified (typically may choose eps=T).

'sigma2',... Gaussian width sigma2*eps for w<<eps (0.5)
'alpha',... shift in log-Gauss broadening
 sigma/4: norm (=height) preserving (default)
 sigma/2: width preserving

'func' input data is not discrete but already function of omega
'nlog',... number of bins per decade of omega scale
 for logarithmic discretization (128 = nlog_fdmNRG/2!)
'emin',... minimum energy for omega binning (1E-8)
'emax',... maximum energy for omega binning (10.)

'disp',... verbose for disp>0 (1)

Note that the first (last) bin contains all data below
(above) emin (emax).

Output

om omega corresponding to the energy binning [-emax,+emax]
As smoothened spectral function

see also fdmNRG.

AWb (C) May 2006

>> getSmoothTDM

Usage: [At [,Info]] = getSmoothTDM(tt, om, Adisc [, opts])
 [At [,Info]] = getSmoothTDM(tt, ARAW, [, opts])

tt	discretized set of time dependent data
om	omega binning for Araw
Adisc	raw TDM data with dimensions [nom, nop [,nnrg]] with nom=length(om), nop=number of operators and, optionally, nnrg the number of NRG iterations
ARAW	alternatively, instead of om and Adisc, the RAW data returned by tdmNRG() in partial mode can be specified as input

Options / Flags

'disc'	discrete data (i.e. does not require d(om) weight; default)
'func'	functional data (i.e. DOES require d(om) weight) default: disc for rank(A)==3 and func for rank(A)==2
'-q'	quiet mode (reduce number of log output)
'alpha',...	exponential decay of energies $\exp(-\alpha dE)$ [0.] this applies alpha in time domain (F. Anders 2005)
'sigma',...	applies broadening in frequency domain (log.gauss) NB! either alpha or sigma can be specified but not both default: sigma=0.1 (with alpha ignored)
'Lambda',..	Lambda from Wilson discretization (required with alpha)
'eps',...	smoothly replaces log-Gauss with regular Gaussian; in absolute units since this routine does not know anything about temperature(!). For the same reason, eps must be specified (typically may choose eps=T).
'nlog',...	number of bins per decade of omega scale for logarithmic discretization (100)
'emin',...	minimum energy for omega binning (1E-8)
'emax',...	maximum energy for omega binning (10.)

Note that the first (last) bin contains all data below
(above) emin (emax).

Output

At	input spectral information transformed into time dependent data.
Info	structure with additional information

see also tdmNRG.

AW (C) Jan 2007

>> **fdmNRG_QS**

Usage: [omega, A0 [,Info]] = ...
fdmNRG(NRG_DATA [,Inrg], B, C, Z0, [,opts])

This routine calculates arbitrary correlation function (CF) in the full density matrix Numerical Renormalization Group (fdm-NRG) framework at arbitrary temperature T for arbitrary abelian or non-abelian symmetries.

$$A(w) = \langle B | | C' \rangle_w = \text{FFT}(\langle \{B(t), C'\} \rangle). \quad (' \text{ equiv } ^\dagger)$$

If B is specified as empty [], it will be automatically assumed equal to C, i.e. B=C as is the case for spectral functions; anti-commutator (default) or commutator will be chosen according to cflags below (see also zflags for Fermionic signs).

This routine uses data from a previous standard NRG run with the result written as a matrix product state in QSpace format. It must contain the state space kept (AK) as well as the one thrown / discarded (AD) at every NRG iteration.

See also: NRGWilsonQS

Input:

NRG_DATA	structure containing data of previous NRG run; if this contains path/file, then the data will be read from the files path/file_##.mat for every site (see NRGWilsonQS)
.AK	MPS state of k(ept) space (regular NRG, QSpace)
.AD	MPS state of t(runcated) NRG space (QSpace)
.HK	effective NRG eigenspectra (kept, QSpace)
.HD	effective NRG eigenspectra (truncated, QSpace)
Inrg	NRG info structure (required if NRG_DATA contains structure; otherwise <NRG_DATA>_info.mat will be accessed. required elements within Inrg structure / data:
.Lambda	NRG discretization parameter for the conduction band. Lambda is needed to undo the energy scaling performed in obtaining the NRG data at the first place.
.EScale	energy scale fore each Wilson shell
.E0	(relative) ground state energy offsets in the same

energy scales as $H[KD]$; this is needed for evaluating the correct weights of the density matrix

B,C operators acting on the impurity used to calculate the correlation between them as in $A(w)=FFT(\langle\{B(t),C'\}\rangle)$.

If the operators B or C are specified by name, they will be read from workspace, and stored in NRG_DATA together with the other NRG MPS data (the latter is disabled by the flag 'nostore')

The operators in B and C are paired up for correlation functions. They are applied in the order of priority (whichever fits first): L, then s, then R onto A_0 as in $\langle NRGDATA \rangle_{00}$ with index order LRs where L = left, R = right = combined, s = local state space sigma).

Z_0 Z_0 operator for the space described by B and C required for fermionic setup where must represent $Z_0=(-1)^{(\text{number of particles})}$; Z_0 is directly contracted onto the B and C where 'zflags' is set.

Options / Flags (default method: fDM)

'fDM'	full-density-matrix approach to CFs (default)
'NRho',k,	build density matrix from single iteration k (for testing purposes, only)
'T',...	temperature at which correlation function is calculated; default: about an order of magnitude larger than the energy scale at the end of the Wilson chain
'vflag',..	more detailed output with increasing value (default: 1)
'partial'	store G1 and G2 separatly (e.g. for detailed balance or the calculation of expectation values)
'nostore'	do not store operator elements of B or C with NRG_DATA
'locRho'	if Rho needs to be calculated it is stored in RAM only and will be deleted after the program is finished
'noRHO'	do not calculate Rho, eg. calculate partition function only (relevant with empty ops B and C only)
'rhoNorm',.	allows to specify the weights rhoNorm explicitly as input (intended for testing purposes only)
'zflags'	on which operators in $\langle B C \rangle$ to apply fermionic signs Z_0 ex. 'zflags', [1 0 ...] would apply Z_0 to the first but not to the second operator (default: all 0 if Z_0)

is empty, 1 otherwise).

'nlog',... number of bins per decade for log. discretization (256)
'emin',... minimum energy for omega binning (TN/1000) ^1)
'emax',... maximum energy for omega binning (10.) ^1)

^1) NB! first (last) bin contains all data below (above) emin (emax)

Output

om	omega corresponding to the energy binning [-emax,+emax]
A0	raw data for spectral function (i.e. discrete in om)
Info	info structure: specific fields are
.ver	version / mode of code
.T	actual temperature used
.rho	FDM weight distribution for given temperature
.RHO	reduced thermal density matrix for A0 (R-basis).
.Om	$\Omega(T) = -T \log(Z)$, (grand)canonical potential.
.A4	individual spectral data for each of the two contributions to the (anti)commutator (hence A4 has twice as many columns as A0 above).
.a4	(1st row) integrated spectral integrals over the two individual contributions to the (anti)commutator of given correlator (2nd row may be ignored; it refers to check w.r.t. detailed balance).
.reA0	contains $\text{Re}(G(\omega \rightarrow 0))$ for the calculated correlation functions. Importantly for finite temperatures, the correlators with $c\text{flags} \neq 0$, i.e. susceptibilities, acquire in addition to the Kramers Kronig transformation (principal value integral!) also a Matsubara correction for $E_a = E_b$, which would be skipped in the principal value integral; this correction [see Hanl and AW, PRB 2014] is added to the first term in the commutator.
.symfac	IRFP factors for given spectral functions.

ALTERNATIVE USAGE

fdmNRG('--mat','nrgdata', 'setup.mat' [, 'B'], 'C')

B and C are the same as above (B=C if B is not specified).

This usage allows to read all input parameters from given MAT files

(MatLab binaries); all variables and options MUST be defined in <setup.mat> by their names shown above; e.g. the operators B and C are specified by their names as defined in <setup.mat>.

The output will be written / updated to the file set '<nrgdata>_##.mat' and '<nrgdata>_info.mat'.

Output will be appended to setup.mat.

See also NRGWilsonQS.

AW (C) May 2006 ; May 2010 ; Nov 2014

>> fgrNRG

Usage: [[[om,] a0], Ifgr] = fgrNRG(NRG0, NRG2, Lambda, B [,C], [,opts])

full density matrix (FDM) Numerical Renormalization Group (NRG) calculation for Fermi Golden Rule (FGR) rates, with extension to resonant inelastic x-ray scattering (RIXS).

FGR mode (default)

This computes the FGR spectral data (om,a0) within the fdm-NRG framework at arbitrary temperature (default $T = T_N^+$ with N the length of the Wilson chain). When Fourier transformed, it corresponds to the time-dependent correlation function [see Ifgr.Itdm.at]

$$a(t) = \langle B(t)C' \rangle = \text{tr}[\rho_0 * B(t) C']$$

Here B and C are local operators acting at or in close vicinity of the impurity. They are assumed to switch back and forth between two disconnected sectors of Hilbert space (e.g. by the presence of an (otherwise unaccounted for) valence core hole). The time evolution $C(t) = \exp(iH_0*t) C \exp(-iH*t)$ therefore becomes 'mixed' with 'initial' Hamiltonian H_0 and final Hamiltonian, iteratively diagonalized for both, NRG0 and NRG2, respectively. The frequency corresponds to $\omega = E_{\text{final}} - E_{\text{initial}}$.

If C is not specified or empty, it is automatically assumed equal to B. This is the typical situation in FGR. The initial state is determined by the full density matrix (FDM) ρ_0 of NRG0 at given temperature (default $T \sim T_N \sim 0^+K$). The dynamics occurs within NRG2.

Input

NRG* (string pointer to) data of previous NRG runs;
if this contains <path/file>, then the data will be
read from the files <path/file>_##.mat files as
generated by NRGWilsonQS.

Lambda NRG discretization parameter used in NRG*.
While EScale and E0 will be read from NRG0_info,
Lambda is needed for simple estimates of energy scales.

B,C local operators, e.g. acting on the impurity. *)
If operators B or C are specified by name, they will be
read from workspace and stored in NRG2_info
(this is disabled by `nostore').

NB! Note that exchanging NRG0 <> NRG2 and B<>C corresponds
to changing from emission to absorption spectra or vice versa.

Options / Flags

'Eoffset',. additional energy offset which affects the omega
binning (om-binning) only. Eoffset acts *on top* of
E0_final-E0_initial, which ensures that by default,
(Eoffset=0) omega=0 corresponds to minimal excitation
energy for a transition from groundstate in NRG0
to groundstate in NRG2 (with NRG2 considered (much)
higher in energy in x-ray absorption).

'T',... temperature at which correlation function is calculated (0.)
'NRho', k builds density matrix from iteration k only (cf. `DM-NRG')

'emin',... min. energy for omega binning (TN/1000) **)
'emax',... max. energy for omega binning (10.)
'nlog',... number of bins per decade for om-binning (256)

'nostore' do not store operator matrix elements of C[12] with NRG data
'locRho' if Rho needs to be calculated it is stored in RAM only
and will be deleted after the program is finished

'sigma',... applies broadening in frequency domain (log.gauss)
NB! either alpha or sigma can be specified but not both.
default sigma=0.3 (with alpha ignored)

'alpha',... exponential decay of energies $\exp(-\alpha \text{ dE})$ [0.]
this applies alpha in time domain (F. Anders 2005)

'partial' return partial data from every NRG iteration
 'raw' raw data only, i.e. no smoothing of data
 'RAW' calculates time dependent data from exact frequency data
 i.e. without binning of energies on log. scale
 'disp' more detailed output (0)

 'Z0',... Z0 operator for first site (impurity) required for
 fermionic setup where $Z0=(-1)^{(\text{number of particles})}$
 'zflags',... on which operators in B.*C to apply fermionic signs Z0
 ex. 'zflags', [1 0 ...] would apply Z0 to the first
 but not to the second operator (default: all 1s,
 as typically the case for absorption/emission setups.

RIXS mode

With the flag '--rixs', this routine computes discrete, i.e., binned,
 RIXS (resonant inelastic x-ray scattering) energy loss spectra
 for given incoming photon energy E_{inc} .
 This uses the Kramers-Heisenberg higher-order generalization of
 Fermi golden rule.

Note on semantics: within RIXS, $i(\text{initial})$ and $f(\text{final})$ refer to the
 same Hamiltonian, whereas it is the intermediate Hamiltonian that takes
 the role of the former $H_2:=H_{\text{final}}$. With

$$V_{if} := \langle i | B \frac{1}{[E_{\text{inc}} + E_i - H_2]} C' | f \rangle$$

the RIXS spectral data would correspond, when Fourier transformed
 into time-domain, to

$$R(t) = \langle V(t) * V' \rangle = \text{tr} [\rho_0 * V(t) V']$$

thus recovering the same final form as for absorption/emission
 spectra above. The RIXS mode requires the additional parameter

'E_inc',... incoming photon energy $E_{\text{inc}} = \text{complex}(\omega_{\text{in}}, \gamma)$;
 this must already also include the broadening γ
 related to finite life-time of the core-hole due to other
 unaccounted for relaxation processes such as Auger etc.

The returned spectra correspond to the RIXS intensity,
 i.e., spectral intensity a_0 vs. energy loss ω .

Output

ω , a_0 binned spectral data (according to $\epsilon_{\text{min}}, \epsilon_{\text{max}}, n_{\text{log}}$)

Ifgr info structure

Notes / comments

- *) Alternatively, if B and C have composite structure that oct on multiple sites/indices simultaneously, the operators B and C can also be specified as cell arrays of QSpace vectors. A given QSpace vector then, e.g., B{i}, is interpreted as the sequence of operators that act like a tensor product on indices $L_{00}, s_{00}, s_{01}, \dots$ where L_{00} and s_{00} are the 'left' and local ('sigma) indices of A_{00} , respectively, and s_{01} , etc. the local indices of A_{01} , etc. In this usage, Z_0 must be already properly included in B and C upon input, i.e. Z_0 and $zflags$ cannot be used.
- **) The bin $a_0(\text{end})$ collects all data with $\omega \geq \omega_{\text{end}} = \omega_{\text{max}}$. For ω_{min} , the smallest bin $a_0(1)$ (given the logarithmic grid!) contains all data where ω is exactly zero (e.g. for diagonal contributions in Lehmann representation). The next smallest bin $a_0(2)$ then contains all data for $0 < \omega \leq \omega(2)$.

Alternative usage:

```
fgrNRG('nrgdata0','nrgdata2','setup.mat','B','C')
```

This reads all input parameters and options from given setup.mat file. Hence the operators B and C must be specified by their name. Output will be written / updated to NRG0 and NRG2, with results also appended to setup.mat.

See also NRGWilsonQS, fdmNRG_QS, tdmNRG

AW (C) 2008 ; RIXS 2019

>> NRGWilsonQS

Usage: `[[NRG_DATA,] info] = ...`

`NRGWilsonQS(H0, A0, Lambda, ff, FC, Z, [, gg, FL, OPTS])`

NRG run using (non-)Abelian symmetries for a Wilson chain of length L based on the standard iterative diagonalization prescription.

The routine performs a quick check whether any of the inputs is complex. This includes the QSpaces H_0 , A_0 , FC , FL (if present), or the hopping amplitudes ff . If so, NRGWilsonQS runs in full complex arithmetic, thus returning complex AK and AD. The subsequent fdmNRG_QS also can deal with either case. All routines use double precision throughout.

H0	impurity / starting Hamiltonian (rank-2 QSpace) H0 will be diagonalized upon initialization hence does not have to be diagonal on input. The resulting basis transformation is multiplied onto A0.
A0	basis used for H0 in LR's order (rank-3 QSpace) where the local space s(igma) must already have the (equivalent) space of a Wilson site (note that A0 is used to obtain the matrix elements of FC at the first iteration.
Lambda	used to rescale H_k by sqrt(Lambda); therefore this should be consistent with the Lambda used to obtain ff and gg below
ff	strength of couplings along Wilson chain as they appear in the physical Hamiltonian, i.e. they must not be rescaled yet, as this is done automatically based on Lambda! if ff is a vector of length L-1, the coupling applies for all nearest-neighbor hoppings via the FC operators in a diagonal fashion. ^1) If ff is a matrix of dimension (L-1) x Nf, the diagonal hopping matrix elements can differ across the FC operators. If ff is a matrix of dimension (L-1) x (Nf*Nf), this also permits to encode cross-couplings FC(i)'*FC(j) in between nearest-neighbor shells. NB! The length L of the Wilson chain is determined by the length of ff, i.e. L=length(ff)+1 by including A0.
FC	set of Nf annihilation operators used for coupling (QSpace) e.g. in case of all-abelian symmetries (default) this becomes $H = F1' * ZF2 + F1 * (ZF2)'$ + h.conj. (zflag=1, see below) with F1=F2=FC such that this operator always appears as a bilinear together with its hermitian conjugate. ^2)
Z	(diagonal) operator required for the calculation of the fermionic hopping matrix elements (QSpace). It is defined by fermionic signs $(-1)^{ns}$ required in the context as described for FC above, where ns is the number of fermionic particles for the local states s(igma).
Options	
zflag	variation of how to apply the fermionic signs Z on the hopping terms in the Hamiltonian determined by the operators FC (default: 1 e.g. as described for FC above in the context of all-abelian quantum numbers). Valid settings:

- 0: ignore Z altogether
- 1: $H = F1'*(Z*F2) + F1*(Z*F2)' + \text{hermitian conjugate (default)}$
- 2: intended for particle-hole-symmetric cases only:
 $H = (ZF1)'*(ZF2)$ at every second iteration only, WITHOUT adding a further hermitian conjugate (already included!) starting with NO Z-operators applied to $A0.local$ (i.e. same as for $zflag=1$), therefore starting with $F1'*F1$ at first iteration (i.e. using F1 to update operators, and again F1 when building enlarged Hamiltonian. For every other iteration, $H = (Z*F1)'*(Z*F2)$ is applied using Z with BOTH operators, consistent with particle-hole symmetry!
- 3: same as $zflag=2$, but Z is applied at every other iteration already starting with the very first iteration.

FL set of Nl local operators along the chain (QSpaces [Nl])

gg strength of local operators FL along the chain similar to ff i.e. not rescaled! (double, $(L-1) \times Nl$, not including $H0$) ^1)
 Note that local operators can always be expressed as Hermitian operators, with real amplitudes gg. Hence gg is always expected to be real.

^1) NB! if the couplings ff (or also gg) are the same for all FC and FL, they may be represented by a single vector only.

^2) FC and FL are considered the same for every iteration throughout the Wilson chain, yet their strength is modified by ff and gg, respectively.

Further options / flags

- 'Nkeep',. max number of states to keep at every NRG iteration (512)
- 'NKEEP',. vector that specifies Nkeep individually for first few sites
- 'Etrunc', use given energy threshold rather than Nkeep to truncate unless Nkeep is reached (≤ 0 uses Nkeep only; default: 0).
- 'ETRUNC', vector that specifies Etrunc individually for first few sites
- 'ET1' Etrunc for 1st with truncation (default: $1.2 \times Etrunc$)
 if $ET1 < 0$, then $ET1 = Etrunc$ is used.
- 'FNfac',. global factor on energy scale, i.e.
 $Escale(n) = \Lambda^{-(n/2)} * 0.5 * (\Lambda + 1) * FNfac$.
 (by default, FNfac is chosen such that $ff(N-1)/EScale(N)=1$;
 see returned info.EScale below)

'Estop',.. stop dynamically once stable fixed point is reached, i.e.
energies EK(1:Nkept/4) for iterations k and k+2 are the same
to within given value for Estop (default: 0, i.e. not used).
'-Estop' same as 'Estop',1E-4

'NEE',.. number of states to store for energy flow diagram (Nkeep*Ns)
'fout',.. store data for every site into output file <fout>_##.mat
if specified, the first RETURN argument is skipped (./NRG/NRG).
'deps',.. restore degeneracy within eps (1E-12)
'db',.. require minimum distance towards discarded energies (-1)
(db<0 searches for maximum distance in vicinity of Nkeep)
'dmax'.. maximum number of extra states to include for db (Nkeep/10)
(mostly a safeguard in case db has been set too large).
'-IdF' build local Id from F-ops, fully ignoring A0 (NB! by default,
sigma-space of A0 is considered a proper Wilson bath site!)
'ionly' info only (incl. Eflow) i.e. do not store iterative NRG data
'-v' verbose mode

Output

NRG_DATA MPS data (only if 'fout' was not specified; otherwise this
data is stored into specified file structure):

.A[KD] states K(ept) / D(iscarded) at current NRG iteration
according to NRG truncation scheme (rank-3 QSpaces in LR's order)
.H[KD] eigenenergies corresponding to the eigenbasis of AK/AD;
stored in diagonal format.

info info structure containing the following fields:

.istr info string
.usage (struct) info such as time stamp, etc.
.HK rescaled energy data for energy flow diagram (QSpace vector)
.EE collected eig(HK) data (purely numerical array)
.E0 subtracted ground state energy at energy scale of iteration
.EK [largest kept, lowest and largest discarded] energy for each iteration
.EScale NRG energy scale used along the Wilson chain (see FNfac above)
.Lambda Lambda used in the calculation (same as input)
.ops (struct) operator specific data for Wilson chain (FC,Z,ff,gg,etc.)
.paras (struct) technical NRG specific internal parameters

Alternative usage: NRGWilsonQS('MATfilename')
NRGWilsonQS('setup.mat' [, 'fout_tag'])

This allows to read all input parameters from given setup.mat file
(MatLab binary) which allows pure standalone run of NRG without

requiring MatLab (except for the API for binary *.mat I/O).

This way all variables and options MUST be defined by their names shown above; moreover, this usage must include a specification for fout_tag as all output data will be written there. fout_tag is either specified as 2nd argument above, else it should be defined in terms of the variable 'fout' in the parameter file setup.mat (default for fout_tag: './NRG/NRG').

AW (C) Apr 2006 ; May 2010 ; Nov 2014

>> tdmNRG

Usage: [cc [,Info]] = ...

tdmNRG(NRG0, NRG2, Lambda, C1 [,C2], tt, [,opts])

time-dependent numerical renormalization group (NRG)
using the full thermal density matrix of the initial system
(FDM), rho_0, as starting configuration at time t=0.

This routine calculates the time evolution of local operators
C1(t)*C2 in fDM-NRG framework at given temperature T.

$$cc(t) = \langle C1(t)C2' \rangle_0 = \text{trace}(\rho_0 * C1(t) * C2')$$

With option -fgr the initial density matrix is modified
by the operator C2 (e.g. creation / deletion of a particle
as in absorption / emission processes) with the
resulting modified time evolution being

$$cc(t) = \text{trace}((C2' \rho_0 C2) * C1(t))$$

Hence the operator C2 appears twice. Note, however, that
the projected density matrix C2' rho_0 C2 is no longer
normalized.

This routine uses data from two previous NRG runs with
the result written as matrix product state in QSpace format.
It must contain the state space kept (AK) as well as the one
discarded (AD) at every NRG iteration.

The initial state is determined by the density matrix rho_0 of
NRG0 at given temperature T. The time evolution is determined
within NRG2.

See also: NRGWilsonQS, dmNRG, fgrNRG

Input:

NRG[02] specifies previously calculated NRG data; if this contains path/file, then the data will be read from the files path/file_##.mat for every site (see NRGWilsonQS)

.AK MPS state of k(ept) space (regular NRG, QSpace)
.AD MPS state of t(runcated) NRG space (QSpace)
.HK effective NRG eigenspectra (kept, QSpace)
.HD effective NRG eigenspectra (truncated, QSpace)

Lambda NRG discretization parameter for the conduction band. Lambda is needed to undo the energy scaling performed in obtaining the NRG data at the first place.

C[12] local operators, e.g. acting on the impurity

If C2 is not specified or specified as empty [], it will be automatically assumed equal to the identity operator.

NB! If operators C[12] are specified by name, they will be read from workspace and stored in NRG_DATA together with the other NRG data (can be disabled by flag `nostore').

Options / Flags (default method: tDM)

'alpha',... exponential decay of energies $\exp(-\alpha dE)$ [0.]
this applies alpha in time domain (F. Anders 2005)

'sigma',... applies broadening in frequency domain (log. gauss)
NB! either alpha or sigma can be specified but not both.
default: sigma=0.3 with alpha ignored.

'NRho', k builds density matrix from iteration k only.

'T',... temperature at which correlation function is calculated.
If not specified, the temperature T is given by the energy scale towards the end of the Wilson chain.

'-fgr' run in FGR modus applying C2 twice (see above).

'nlog',... number of bins per decade for logarithmic discretization (256)

'emin',... minimum energy for omega binning (TN/1000)

'emax',... maximum energy for omega binning (10.)

'nostore' do not store operator elements of C[12] with NRG_DATA

'locRho'	if Rho needs to be calculated it is stored in RAM only and will be deleted after the program is finished
'partial'	return partial data from every NRG iteration
'raw'	raw data only, i.e. no smoothing of data
'RAW'	calculates time dependent data from exact frequency data i.e. without binning of energies on log. scale
'disp'	more detaild output (0)
Z0	Z0 operator for first site (impurity) required for fermionic setup where $Z0=(-1)^{(\text{number of particles})}$
'zflags'	on which operators C[12] to apply fermionic signs Z0 ex. 'zflags', [1 0 ...] would apply Z0 to the first but not to the second operator (default: none).

NB! The first (last) bin contains all data below (above) emin (emax)

Output

cc	expectation value at times specified in tt
Info	info structure

See also NRGWilsonQS, fdmNRG_QS, fgrNRG

AWb (C) Jan 2007

3 MEX routines (./util folder, selected)

>> matchIndex

Usage: [Ia,Ib,I] = matchIndex(QA, QB [,OPTS])

QA or QB are matrizes whos rows are matched; consequently, their number of columns (row dimension) must be equal.

Options

'-s' sort indizes with respect to A
'-q' quiet mode, i.e. do not issue warning when NaN's are encountered.
'eps',...specify epsilon within which to accept match (0)
'-f' use float instead of double (gets rid of numerical noise in case of double precision input data)

Output

Ia index into QA (QA.Q)
Ib index into QB (QB.Q)

I an info structure with elements
.n number of matches [=length(Ia)]
.dim number of input records [size(QA,1), size(QB,1)]
.ix1 entries in QA not matched in QB
.ix2 entries in QB not matched in QA

AW (C) 2006 ; Feb10,11

>> mpfr2dec

Usage: [data vector]=mpfr2dec([char matrix] [,base]);

Convert mpfr data strings to decimals.
Since matlab is col-major, the data string for each value is expected to represent an entire column.

Example

```
mpfr2dec(['0':'9', 'A':'Z', 'a':'z'],62)
```

Wb,Jul01,14

>> wbrat

Usage: [q,I] = wbrat(q [,opts]);

get rational approximation of given number
based on continued fraction approach.

Options

'-h' display this usage
'-v' verbose mode
'-q' quiet mode
'~r' do not include square root check [i.e., using rat() on q^2]
'~s' do not return string, but rath numbers q=P/Q
'-a' also return continued fraction sequence in I
'nmax',... maximum number of iterations (6)
'eps1',... tolerance on continued fraction (1E-6)
'eps2',... rel. overall tolerance wrt. to input number q (1E-12)

Output

q same as input, but 'fixed' to rational approximation
I info structure
 .P integer enumerator
 .Q integer denominator
 .relerr relative error of rational representation
 .n total number of continued fraction iterations used
 .param (input) parameters nmax, eps1, eps2 used

See also MatLab's rat.m

AWb (C) July 2010

>> helloworld

Hello world from MEX! (.mexmaci64: nargout=0, nargin=1)

>> mexworld

function mexworld [opts]

This is a test function for mex-files.

Options

--check-hl check color output for wblog entries as used by MEX files
--check-env checks wblog-related environmental variables

Wb,Jan12,19

4 @QSpace class member functions (Matlab)

>> acomm

```
function C=acomm(A [,B,cflag])
```

cflag if set, must equal 'conj'
and is interpreted as conjA, i.e. $C = \{A', B\}$

Wb, Feb06, 16

>> addSymmetry

```
function A=addSymmetry(A,qtype [,opts])
```

Append scalar symmetry (e.g. new symmetry label,
where all existing states get a newly added qlabel $q=0$
with the zeros repeated according to the rank of the
added symmetry.

Options

'pos',... actually insert symmetry at particular position
in symmetry order; e.g., 'pos',1 will prepend the symmetry.
'q',... specify particular symmetry labels other than (default $q=0$).
if non-zero, this assumes A to be of operator type in that
specified q is applied to the first two indices only *)

*) rather than specifying non-zero q, consider starting with zeros
first, followed by mapping of operators into tensor product
of state spaces of operators

See also: QSpace/makeIrop

Wb, Jul12, 11 ; Wb, Dec08, 14

>> appendSingletons

```
function A=appendSingletons(A,'new_itag[*]')
```

```
function A=appendSingletons(A,qdir)
```

```
function A=appendSingletons(A,r_new,qdir)
```

Add singleton dimension(s) to reach rank r_{new} .
qdir sets direction of new singleton dimensions (numeric; e.g. +/-1)

In usage #1 assigns the specified itag to the newly added singleton

dimension; this also permits to simultaneously specify qdir via a trailing conjugation flag '*'.

If numel(qdir)>1, it is assumed it specifies qdir for all of A where the values in qdir are ignored for the ones already existing in A [this is useful if qdir was obtained via getqdir(Aother)].

NB! use makeIrop() to append irop dimension instead of this, since there may be non-zero symmetry labels for all-abelians ops.

Examples

```
A4=appendSingletons(E,' +-')
```

See also makeIrop, fixScalarOp.
Wb,Sep17,18

>> cat

Usage #1: X=cat(A1,A2,...,k)

Concatenate data sectors of given QSpaces along dimension k (QSpace vectors are also included in the concatenation procedure). NB! k may be a cell, i.e. {k,qdir} to specify qdir of newly added dimensions.

Usage #2: X=cat(dim,A1,A2,...)

This replicates the kefault matlab behavior of cat(dim,A,B,...). Consistently, therefore dim is also listed as first argument, vs. k ~ dim as last argument in usage #1.

Wb,Sep17,18

>> checkOM

```
function m=checkOM(A [,opts])
```

Check for outer multiplicity in given QSpace

Wb,Feb15,14

>> checkSameSpace

```
function i=checkSameSpace(A,ia,B,ib)
```

check whether QSpaces A and B share the same vector spaces (simple check in terms of symmetry blocks and respective multiplet [and Clebsch-Gordan dimensions, if applicable]).

This routine uses getQDimQS(A,ia) [and similar for B] hence ia accepts the same index specification as getQDimQS that is, a specific dimension or 'op'(erator).

Wb,Jan12,12

>> chopd

```
function A=chopd(A [,aref])
```

This 'chops data' in A.data{} in the sense that it sets small matrix elements below threshold to zero, and removes imaginary noise if any. The numeric threshold used is based on $\text{eps} = \text{aref} \times 1\text{E-}12$ where by default $\text{aref} = |A|$.

Wb,Oct17,11

>> comm

```
function C=comm(A [,B,'conj','+'])
```

Compute commutator (or anticommutator, if '+' is specified).
if B is omitted, this assumes $B=A$ && 'conj'.
Here 'conj' is interpreted as $\text{conj}A$, i.e. $C = \{A',B\}$
For anticommutator, use $\text{comm}(A [, B, \text{cflag}], '+')$;

The operators A (and B) must be rank-2 or rank-3, where the latter case assumes operator index order (ss',o), i.e. the irop (=spinor) index comes last.

Examples

```
comm(F,'+') computes {F',F} // same as acomm(F)
```

Wb,Feb06,16

>> conj

function: A=conj(A)

get complex conjugate of operator in QSpace format;
overloading the conj routine

Wb,Jun02,08

>> contract

function A=contract([args])

same as contractQS
but returns QSpace object (rather than plain structure).

Wb,Sep18,06 ; Wb,Mar31,16

>> ctranspose

function A=ctranspose(A)

This overloads the hermitian conjugate operator '
(accepts hyperindex)

Wb,Nov30,09

>> datasize

function [D,DD]=datasize(A ['-s'])

get full listing of size(data{ }),
except if option '-s' is specified which gives summary
in terms of total size in bytes of A.data

Wb,Apr24,08

>> decomposeOp

function [cc,err,I]=decomposeOp(H,{01,02,...}[,R][,opts])

decompose operator H into operator (sets) specified in X={01, 02, ...};
if R is specified, operator overlap is calculated
w.r.t. density matrix R.

Wb,Apr16,13

>> deconj

function [A,t]=deconj(A [,idx])

Unset "bra" (complex conjugate) label for specified dimensions
(all if idx is not specified).

Wb,Apr17,13

>> diag

function: A=diag(A [,OPTS])

Toggle that switches between diagonal formats similar to matlab's behavior of diag(): for full operator A, extract diagonal of A in QSpace format; if already in diagonal format, diagonals become restored to full format again.

Options

- d return numeric data vector of diagonal only
- t store diagonal as rows (by default,
diag(M) of matrix M puts diagonal of M into column)
- c compact diagonal QSpace to diagonal only
this throws an error if non-diagonal data is present in A

Wb,Sep08,06

>> dim

function s=dim(A [,opts])

Options

- '-a' all (simply return output from getDimQS())
- '-f' full dimension
(in case of CGC spaces, corresponds to state space dimension)
- '-op' IROP dimension (i.e. checking for rank-3 QSpace)

Wb,Jul31,12

>> display

function display(A [,OPTS])

Options

- a, -f show all record entries even if many (-f like full, force)
- c compact mode (e.g, shows combined CGS dimensions only)
- v verbose flag (in case of QSpace array, show detailed content for all entries)

- E sort QSpace wrt. data (energy) [assumes diagonal Hamiltonian]
- s sort QIDX
- 'sperm',... sort QIDX using given permutation

By default, QSpaces with many entries (length > 14) will show a truncated list, including also the two largest entries.

Wb,Mar01,08

>> eig

```
function [varargout]=eig(H,varargin)
    ee=eig(H,varargin);
    [A,E]=eig(H,varargin);
```

Wb,Aug30,12

>> eq

```
function i=eq(A,B)
```

Simple wrapper to QSpace/isequal.m (whether two QSpaces are equal) that overloads Matlab's the == (equal) operator.

Wb,Feb22,24

>> finditag

```
function i=finditag(A,pat [,n])
```

find itag with specit regexp pattern
if n is specified, the result must yield exactly n matches.

Wb,Jun03,19

>> fixAbelian

function fixAbelian(A,B,...)

Options

- xop3 reduce "scalar" rank-3 operators to regular rank-2.
- addCGR add (trivial) CGR space to all-abelian operators.
- op3 together with -addCGR: make operator of rank 3 if rank 2.

Wb, Feb20, 13

>> fixScalarOp

function fixScalarOp([opts,] A,B,...)

reduces "scalar" rank-3 operators to regular rank-2.
converting (yet keeping) the Clebsch Gordan coefficient
spaces to sparse.

Options

'-f' enforce reducibility if rank-3.

See also makeIrop() to append 3rd dimension

Wb, Aug21, 13

>> get_qdim

function [dd,D]=get_qdim(A,dim)

get cgr dimension for given dimension

Wb, Nov27, 15

>> getAtensorLoc

function A=getAtensorLoc(X [,qvac][,l])

get Atensor by extracting local state space from local
operator X together with vacuum state on virtual bond
(default for qvac; optionally any other bond-space can
be specified as third argument, instead).

l \in {'L','R'} specifies the location of vacuum state

using LR's index order (default: l='L').

All arguments except for the last (l) are directly handed over to `getIdentityQS()`; hence this also allows more general QSpace tensors X and qvac together with the explicit specification of the space index to use.

Wb,Dec28,14

>> getEQdata

```
function [E,Q,D]=getEQdata(HK [,opts])
```

Wb,Sep22,12

>> getId

```
function A=getId(A,qs [,k])
```

reduce QSpace A to rank-2 Id tensor which only has vacuum state / scalar representation with each index. if k is specified, itag{k} will be used.

Wb,Oct18,14

>> getIdentity

```
function A=getIdentity([args])
```

Wrapper routine to `getIdentityQS` that returns QSpace object rather than plain structure.

This wrapper supports a tweak in the trailing itag for the new fused state space: if represent, it can be specified as cell string { 'tag', 'x' } with some extensions string 'x'; in this case trailing marks such as ['*] with 'tag' are moved to the end of 'x' instead, prior to concatenating them.

Wb,May28,16

>> getIdentity3

```
function E=getIdentity3(A,idx [,tag,opts])
```

Map pair of open indices to be combined as in δ_{ij} into single index in $E_{(i,j,k)}$ by mapping $(i,j \rightarrow k)$. The added third index therefore always transforms like a scalar for all symmetry sectors present, i.e., has q-labels all zero.

Options

--rho normalize E
such that it resembles density matrix in (i,j) for fixed k.
tag tag assigned to the new trailing index.

Wb,Aug28,20

>> getitags

```
function [t,conj,mark]=getitags(A [,it])
```

return itags as cell array, except if single itag is requested via the index set $it = \text{index of tags}$. In the later case, if two additional arguments are requested, trailing conj-flag (*) and marker (') are returned as counts in cflag and mark, respectively.

adapted from QSpace/gotITags.m

Wb,Apr10,14

>> getproj

```
function P=getproj(H [,eps,opts])
```

get projector with respect to H into ground state space (diagonalize if necessary); eps determines the window wrt. ground state energy, within which state are kept ($1E-8$).

Options

'-q' quiet mode

Wb,Aug30,12

>> getqdim

```
function [dd,ss]=getqdim(A)
```

get symmetry specific length of q-labels

Wb,Sep18,11

>> getqdir

```
function t=getqdir(A ['-s'])
```

qdir in numeric format (+- => [+1,-1])

NB! input arguments are directly handed over to getitags.m

Options

'-s' return qdir as string (default: numeric; see above)

Wb,Mar30,15

>> getqfmt

```
function fmt=getqfmt(A [,opts])
```

get format string for a single $Q\{i\}(j,:)$

Wb,Dec14,15

>> getQRange

```
function qq=getQRange(A,k)
```

Get range of symmetry sectors in specified QSpace A.

Wb,Feb24,08

>> getQSpectra

```
function [q,EE,qq,dz,DZ]=getQSpectra(H,ws)
```

get symmetry-resolved energy spectra.

Input

H diagonal scalar operator (with diagonals stored only)
ws specifies which symmetry to pick

Output

q unique set of labels for symmetry ws
EE cell array of energies, one cell for each symmetry in q
qq full set of symmetry labels (expanded to same size as EE)
dz full set of multiplet degeneracies
DZ full set of multiplet degeneracies (expanded to same size as EE)

Wb, Apr12, 12

>> **getscalar**

function x=getscalar(A)

Assuming that A is the result out of full contraction,
this results in a scalar that can be extracted here
with safety checks.

Wb, Aug28, 08

>> **getsub**

Usage #1: function [A,B]=getsub(A,idx)

Select given subspace where idx represents direct index
into records of QSpace.

Usage #2: function [A,B]=getsub(A,q,dim)
function [A,B]=getsub(A,{q} [,dim]) // Wb, Feb21, 21

Select subspace that has given symmetry sector(s) in given dimension(s).
if q is a cell array, then this directly relates to dimensions in A
e.g. getsub(ops,{[],[],0}) = getsub(ops,0,3) picks scalar sector
on 3rd dimension; here '0' or 0 can be used as a shortcut to zeros(1,m)
in q, with m the number of symmetry labels.

Usage #3: function [A,B]=getsub(A,'-0') // Wb, Dec04, 20
function [A,B]=getsub(A,'-0!')

Assuming A is an irop (rank-3 with irop-index on 3rd leg)
select scalar operator part (q-labels 0); this is equivalent

to usage #2, having `getsub(A,[0 0 ..],3));`

in case of '-0!', also remove singleton irop dimension
(in case it is not a singleton, i.e., by having multiple
q=0 multiplets, an error is issued)

Options

-d return A.data (rather than A itself)

If second output argument is requested, this will contain
whatever was left in A_in as compared to what was selected
in A_out.

Wb,Aug20,08 ; Wb,Jun19,13

>> getsym

function [ss,s]=getsym(A [,dim,opts])

Options

-c return symmetries split as cell array
-d get dimensions (rank) of each symmetry [returns 1 for Abelian]
-r get symmetry-rank for each symmetry [returns 0 for Abelian]
'ir',dim index range in q-labels for given dimension
'-I',dim get extended info structure for given symmetry

Wb,Oct24,11

>> getvac

function A=getvac(A [,k])

reduce QSpace A to rank-2 Id tensor which only has
vacuum state / scalar representation with each index.
if k is specified, itag{k} will be used.

Options

-1d reduce to rank-1 vector (NB! only permitted for
for scalar representation with all q-labels zero!)

Wb,Oct18,14

>> getzdim

```
function [dd,qq]=getzdim(A [,dim,opts])
```

get multiplet dimensions of QSpace A.

Options

- p product of multiplet dimensions
- x expand to match data (repmat)
- u get dimension of unique qlabel set (returned as 2nd output argument)

The specification of the dimension (dim) may also take the following string values:

- '-op' gets dimension of IROP set
- 'op' gets dimension of state space dimension for scalar operator
(together with '-u' this is equivalent to 'op') // deprecated

Wb,Jan24,11

>> gotCGS

```
function [i,rsym] = gotCGS(A)
```

Whether got Clebsch-Gordan reference space (CGS)
where rsym specifies the rank of the symmetries.

Wb,May16,10

>> imag

```
function A=imag(A)
```

extract imaginary data of operator in QSpace format

Wb,Oct19,11

>> info

```
function info(A [,istr,cflag,rmax])
```

short general info on QSpace
e.g., used as QSpace header in display().

Options

istr info string
cflag whether to show info in compact form ('-c','-C')
rmax max. rank to consider (used to reserve space for itags
so that entries line up over repeated calls e.g.
for a QSpace array.

Wb,Mar01,08

>> info3

function info3(A,dim [,opts])
graphical analysis of size distribution of CGC spaces

>> isAbelian

function i=isAbelian(A [,opts])

Check whether A has Abelian symmetries only.

Wb,Feb20,13

>> isdiag

function [isd,istr]=isdiag(A [,opts])

check whether QSpace is in diagonal rank-2 format.
The return value isd is as follows:

- 1 if block-diagonal in symmetry space (here blocks themselves
do not need to be diagonal see option -f to also enforce this)
 - 2 if on top of (1), all data{} sets are row-vectors,
e.g., compressed format as returned bei eigQS()
 - 3 same as 2, but if all entries in data{} are column-vectors.
- 0 if none of the above.

Options

'-d' If all data{} blocks are scalars, i.e., of dimension 1x1,
by default, this returns isd=1; with '-d' this returns
isd=2, instead.

'-f' check for fully diagonal, i.e., also enforce diagonal blocks data{} within numerical noise (1E-12) for return value 1 (if non-diagonal, this returns 0, instead).

Wb, Apr24, 09

>> isempty

function i=isempty(A)

Check whether A is an empty QSpace (e.g. has no non-zero blocks).

Wb, May12, 06; Wb, Aug23, 08

>> isequal

function i=isequal(A,B [,opts])

overloading isequal and ==
where input A or B can also be struct(QSpace object).

Note that the usage A==B is equivalent to isequal(A,B),
i.e. this assumes the default setting without any options.

Options

'-l' lenient (ignore difference in itags)
'-f' enforce all fields to be the same (no lenience at all)

Wb, Nov09, 09 ; Wb, Jun28, 21

>> isHConj

function q=isHConj(A [,deps])

Check whether input A represents Hermitian conjugate operator within threshold deps (1E-12).

Wb, Apr04, 23

>> isop

function is=isop(A)

Wb, Nov25, 11

>> isOpConj

```
function i=isOpConj(A [,output_type])
```

Check whether input A is compatible with irop structure
in which case the direction of the irop index is returned
(with optional 2nd argument, a string is returned, instead).

Wb,Jun06,19

>> isOrtho

```
function a=isOrtho(A)
```

This routine looks for single outgoing index. If found,
it returns the location of this leg within the index order,
or zero otherwise.

Wb,Jan15,15

>> isreal

```
function i=isreal(A)
```

checks whether A contains complex data
(this overloads matlabs isreal() function).

Wb,Apr23,15

>> isscalar

```
function i=isscalar(A [,opts])
```

Checks whether QSpace is a scalar, meaning rank-0
with a single number in data [e.g., a scalar is obtained
by tracing/contracting all indices resulting in a single
number].

Options

- l lenient (also consider a QSpace with non-empty Q
a scalar, as long as data contains a single number)
- x extended mode (in case of input QSpace array,
show isscalar() for each individual QSpace

Wb,Aug04,08

>> isscalarop

```
function [s,dd]=isscalarop(A [,opts])
```

requirements for scalar operators:

- (1) rank-2
- (2) block-diagonal in symmetries
- (3) all cgc spaces are identities

Options

-1 lenient (also accept rank-3 operators)

See also QSpace/issingleop for non-scalar, yet still rank-2 operators.
Wb,Jul05,12

>> issingleop

```
function [s]=issingleop(A [,opts])
```

check for possibly non-scalar, yet still rank-2 operators.

See also QSpace/isscalarop for Scalar, i.e. block-diagonal (rank-2) operators.

Wb,May11,13

>> issingleton

```
function i=issingleton(A,kk)
```

Wb,Apr24,08

>> itag2iter

```
function k=itag2iter(A)
```

extract site position to k from A.info.itags
with expected itag format '[non-number]##[non-number]'

Wb,Aug01,17

>> itagrep

```
function A=itagrep(A [,idx], )
```

Apply given regexprep() pattern to itags
specified by idx (default: all).

Wb,Jan14,15

>> itags2int

```
function k=itags2int(A)
```

Extract integer indices carried with itags,
assuming the itag format '[non-number]##[non-number]'
where outgoing indices (i.e., which carry a trailing
conjugate flag `*') are returned as negative integers.
If no valid number is encountered, nan is returned
for that index.

Wb,Nov19,21

>> itags2odir

```
function odir=itags2odir(A [,opts])
```

find index that points outwards of A (outgoing index;
expecting single outgoing index, by default).

Options: see usage for private/itags_to_odir.m

Wb,May15,17

>> itags2str

```
function s=itags2str(A [,use_col])
```

return combined itags as string

Wb,Jul13,23

>> logical

function q=logical(A)

Overload of conversion to logicals (unary operator)
e.g. for conditionals: if A, ...; else ...; end

see also QSpace/ [which returns ~logical()]
Wb, Feb18, 21

>> makeIrop

function A=makeIrop(A [,t_op])

make A an irreducible operator with a proper 3rd irop index,
which thus is only intended for rank-2 scalar operators.
Operators already in rank-3 format are checked for the
right format.

Options

t_op specifies itag for the irop leg, only if added
(empty by default).

See also: fixScalarOp, appendSingletons.
Wb, Feb12, 16

>> mapQ

QM = Q map that transforms Q numbers to 'rotated' set
Example: SIAM: (N1,N2) --> (Q,Sz)

$$\begin{array}{ccccc} \text{QM} & * & \text{Q_old} & + & \text{offset} & = & \text{Q_new} \\ \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} & * & \begin{bmatrix} \text{N1} \\ \text{N2} \end{bmatrix} & + & \begin{bmatrix} -1 \\ 0 \end{bmatrix} & = & \begin{bmatrix} \text{Q} \\ \text{Sz} \end{bmatrix} \end{array}$$

Wb, Aug10, 06

>> markitags

function A=markitags(A,idx)

mark specified indices in idx for QSpace A.

Wb, Aug12, 23

>> minus

```
function C=minus(A,B)
```

This overloads the - operator in Matlab.

Wb, Apr24, 09

>> mrdivide

```
function C=mrdivide(A,B)
```

This overloads the right-divide operator.

It is restricted to numbers B only, and hence permits the syntax A/x for QSpace A and x a numeric value.

Wb, Sep28, 12

>> mtimes

```
function C=mtimes(A,B [,ia,ib])  
overloading * operator (matrix multiply)
```

Options ia and ib are only relevant, if this routine is explicitly called as mtimes(..). By default, this assumes operators, ia=2 and ib=1 (matrix multiply).

Alternative usage

```
function C=mtimes(A,B,'--op')
```

In case that both A and B are irops (rank-3 operators), by default, the routine above contracts(!) the operator index, assuming it can be contracted, i.e., S^*S also performs the dot product on the irop index.

In order to prevent this, the option '--op' keeps the operator indices open, and if present on both A and B, contracts them into a single irop index. If the operator indices have different directions, a 1j symbol is inserted for this (therefore note that for non-trivial 1j symbols such as for half-integer spins, this defines the sign of the operator overall. This way, for example, `mtimes(F1',F2,'--op')` keeps the irop index.

Wb, Jul04, 12 ; Nov19, 23

>> norm

```
function M=norm(A)
```

Simple wrapper routine for normQS().

Wb,Oct18,14

>> norm2

```
function M=norm2(A)
```

Wrapper routine norm2 -> normQS().^2

Wb,Oct18,14

>> not

```
function q=not(A)
```

Overload of conversion to logicals (unary operator)
e.g. for conditionals: if ~A, ...; else ...; end

Wb,Feb18,21

>> numsym

```
function ns=numsym(A)
```

determine number of symmetries in QSpace(s) A.

Wb,Feb06,14

>> olap

```
function x=olap(A [,B,opts])
```

calculate frobenius norm / overlap.

Wb,May27,11

>> oplus

```
function C=oplus(A,B [,d12 [,opts]])
```

Direct sum (latex oplus) of two tensors A and B along dimensions d12 in a block-diagonal sense (default: d12=[1 2]). If d12 is single index, oplus is performed along that dimension only. For example, the following are equivalent

```
oplus(A,B,2)
oplus(A,B,'first')
oplus(A,B,[1 2,'first'])
```

Options

'first' direct sum only in index d12(2)
'last' direct sum only in index d12(1)
 where first/last is interpreted from an MPS point of view, where the first (last) matrix shall be extended along the 2nd (first) dimension, respectively.
 By default: direct sum in (both) indices in d12.

'bfac',... apply bfac to B as in: $A \oplus (bfac \cdot B)$

E.g. see <https://mathworld.wolfram.com/MatrixDirectSum.html>
Wb,Nov23,20

>> ortho

```
function [A1,A2,Io]=ortho(Psi [,opts])
```

Plain wrapper routine to orthoQS()

Wb,Sep05,23

>> permute

Usage: A=permute(A,p)

Permute legs of given QSpace with permutation p (simple wrapper to permuteQS).

Wb,Sep08,06 ; Wb,Nov24,14

>> plotEdata

```
function plotEdata(H [,opts])
```

plot eigenspectrum of given QSpace,
expecting scalar operator upon input!

see also QSpace/
Wb, Feb15, 13

>> plotQSpectra

```
function Iq=plotQSpectra([H|R] [,opts][,SOP])
```

graphical analysis of eigenspectrum of Hamiltonian
(or entanglement spectrum of density matrix; requires '-ES' flag)
separating the data into its symmetry subspaces.

Options

'-ES' entanglement spectrum (expects Rho as input)
'-x' show state space dimensions (rather than multiplet dimensions)

'yl',... ylim range
'y2',... y-range within which degeneracies are indicated
'ws',... which symmetries to pick (e.g. [1 2] for the first two)

SOP more verbose description of symmetry labels, which also
 appears in the title of the respective panels.
 SOP may simply be specified through IS.SOP, i.e. as returned
 by getLocalSpace() in the NRG setup; all that is used, however,
 are the strings SOP(:).info, hence, alternatively, SOP may
 only contain {SOP.info}. keywords looked for are 'spin' or
 'charge' which effects the xlabel.

Adapted from getrhoESpectra.m
Wb, Apr11, 12

>> plus

```
function C=plus(A,B)
```

This overloads the + operator in Matlab.

Wb, Apr24, 09

>> QSpace

```
function [A,varargout] = QSpace(...)
```

Construct QSpace class object.

This serves to wrap QSpace structures into QSpace objects, and also can be used to generate several specific QSpace objects. For historical reasons, this also includes low-level QSpace constructors for the case of only abelian quantum numbers.

Usage:

```
#1: A = QSpace(Q,data)
    A = QSpace(A)    conversion e.g. from struct
    A = QSpace(A,'check')  also performs full check on QSpace structure

#2: A = QSpace(q1, data1, q2, data2, ...) % see initQSpace

#3: A = initQSpace(Q1,M,DB)
    square operator M with row/col Q1 block size given by DB

#4: A = initQSpace(Q1,Q2,M,'map')
    matrix M with row/col QIDX given by Q1/Q2
    eg. used for projectors

#5: A = QSpace(Q,M,'operator');
#6: A = QSpace(A,'unity');

#7: A0=initQSpace(QL,Qs1,Qs2,...); setup identity tensor
    for product spaces (QL,Qs1,...)

    it must hold: size(QL,2) = size(Qs1,2) = size(Qs2,2) = ... = QDIM.
    Qs and QL span the full Hilbert space, hence certain quantum
    labels may appear several times.
```

Wb,May12,06 ; Wb,Apr16,16

>> randomize

```
function A=randomize(A [,nrm,opts])
```

Randomize A.data using randn()
while preserving overall norm (unless explicitly specified).

Options

-z use complex random numbers (default: real)

Wb,Nov11,09

>> rank

function r=rank(A)

Returns the tensor-rank of QSpace A, i.e., its number of legs.

Wb,Mar02,08

>> real

function A=real(A)

extract real data of operator in QSpace format

Wb,Oct19,11

>> rmitags

function A=rmitags(A)

skip itags of given QSpace(s)

*** NB! deprecated; rather use untag ***

Wb,Apr14,16

>> rmtrace

function A=rmtrace(A)

remove trace from input tensor A

Wb,Apr24,13

>> sameas

```
function i=sameas(A,B [,eps=1E-10][, '-l'])
```

check whether QSpaces A and B are the same
except for minor differences such as
info.cgr.cid(end) and info.cgr.nnz

Options

eps data is the same to within eps (default: 1E-10)
-l permit different order entry (default: no)

Wb, Aug08, 16

>> SEntropy

```
function [se,rr,I]=SEntropy(Rho [,eps,opts])
```

Calculate von Neumann entropy of the mixed (reduced)
density matrix R.

R may also contain non-abelian symmetries in which case
the output weights rr are already weighted by their
corresponding multiplet degeneracies such that $\text{sum}(rr)=1$.

Options

eps check $\text{trace}(Rho)=1$ within numerical accuracy of eps (1E-12)
'-xm' expand multiplet degeneracy in returned rr
'beta',... assume R is a Hamiltonian => apply $\beta=1/T$ and exponentiate

Wb, Jan24, 11

>> SetCC

```
function [A,t]=SetCC(A [,idx])
```

Set "dagger" for specified index-labels (all if idx is not specified).
If any $A.data\{k\}$ is complex, A.data will also be complex conjugated.

Wb, Aug04, 12

>> setitags

function A=setitags(A,...)

set itag(s) while preserving conj flag with usage #1 - #6
as detailed below. Note that conjugate flags always remain
preserved, i.e., are ignore in the input arguments if present.

Usage #1: A=setitags(A, [{i1,i2,...},] tt) ^1)

set specified leading set of, or all itags tt = {t1,t2,...}
setitags(A,{'K','K','s'},{1,2,2}) using cell, numbers are appended to itags
setitags(A,{'K1','K2','s2'}) same as above, specifying explicit itags

Usage #2: A=setitags(A,i,tt) ^1)

set itags for indices i to specified itags tt = {t1,t2,...}
(note that this differentiates from usage #1 by using a
numeric index array i, rather than a cell array with usage #1).

Usage #3: set itags for A-tensors

MPS-like basis transformations with index order convention
LRs = (left,right, local state space) in mind

A=setitags(A,'-A',k) {'K<k-1>','K<k>','s<k>'}
A=setitags(A,'-A:K,K,s', k) {'K<k-1>','K<k>','s<k>'} more specific
A=setitags(A,'-A>K,K,s@n',k) same, also using n-digit format (default: 2)
A=setitags(A,'-A<K,K,s@n',k) {'K<k>','K<k+1>','s<k>'} i.e. R->L order

Usage #4: set itags for operators

with index order convention X,X',op = [bra,ket,irop index]

S=setitags(S,'-op:X','op') {'<X>','<X>','op'}
S=setitags(S,'-op:X[,op[#]][@n]',k) {'<X><k>','<X><k>','op[k]'}
- where the default operator itag if not specified is 'op'
- if operator itag contains trailing #
the index k will be added in the same format as with X
- the optional trailing '@n' specifies n-digit format (default: n=2)

S=setitags(S,'-op:<regex>', A) set itags based on behavior
that mimicks contractQS

Usage #5

A=setitags(A,ia,B,ib); copy itags from B to A

Usage #6: any of the usages above, but return the final itags rather than A, by requesting 2 output args, e.g.

```
[tt,i]=setitags(A,...)
[tt,~]=setitags(A,...)
```

this returns itags+indizes themselves, rather than setting the itags in A (the conjugation is inherited from A; hence only a single QSpace A is expected).

^1) the order of arguments for usage #1 and #2 changed switching the 2nd and 3rd argument for more intuitive reasons; therefore for backward compatibility [07/05/2023]: usage #1 and usage #2 also support the reverse order of arguments 2 and 3, i.e., permits one to specify the index (set) i last.

Wb,Mar24,16 ; Wb,May28,18

>> SetOL

```
function A=SetOL(A,X,k [,otag]) % usage #1
function A=SetOL(A,itag[,otag]) % usage #2
```

Set operator labels of the type { itag, itag* [, 'op*'] }.
default operator tag (otag) for rank-3 tensors is 'op'.

Usage #1

set index labels for operator A based on the label found in tensor X at dimension k.

Wb,Aug04,12

>> showdeg

```
function [I]=showdeg(H [,opts])
```

show degeneracy of given rank-2 operator, either in diagonal representation, or to be diagonalized.

Options

'dE',... considere values degenerate within dE (1E-3)
'ng',... number of degenerate groups to display

'-f' force flag
'-v' always print lowest part of spectrum
'-V' more verbose still.

Wb,Apr12,11

>> **skiprecs**

function A=skiprecs(A,ix)

Remove QSpace records ix from QSpace A.

Wb,Aug01,12

>> **skipzeros**

function A=skipzeros(A [,eps,'--all'])

Skip records in A with negligible weight (based on Frobenius norm).
See skipZerosQS for more information.

Wb,Nov10,07

>> **sort**

function [A,isd]=sort(A,perm)

sort records in QSpace according to q-labels after using
permutation perm (default: Id). Alternatively, perm may
also be specified as

'-E' (sort by energy) or
'-R' (same as '-E', but sorts in descending order).
'-S' sort wrt. to size of data+cgr

Wb,Sep12,06

>> **split_ops**

function [A,nsp]=split_ops(A [,opts])

split composite operators back to (irreducible) operator sets
that have well defined irop symmetry labels
(returns A -> A(:) expanded along dim2, such that

A(k) still references a (partial) original QSpace).

Options

--set-op	set as operator type
--s2f	move scalar operators to front
--trans	return transpose array
--vec	return as serialized vector
--cell	return as cell array

Wb,Aug31,19

>> **sqrt**

function B=sqrt(A)

Compute sqrt(A) for diagonal QSpace(s) A
(error is issued if A is not diagonal)

Wb,Mar11,19

>> **squeeze**

function [A,I]=squeeze(A [,OPTS])

remove singleton dimensions (similar to matlab's routine)

Options

'-start' only singletons at the front
'-end' only singletons at the end (default: all)

Wb,Apr24,08 ; Wb,Dec01,20

>> **subsasgn**

function Aout=subsref(A,S)

This manages setting fields
in the underlying structure of QSpace A.

AW (2007, 2012)

>> subsref

```
function Aout=subsref(A,S)
```

This manages access to the fields
in the underlying structure of QSpace A.

AW (2007, 2012)

>> sum

```
function S=sum(A,dim)
```

This overloads the sum operator using plusQS().
The second argument acts similar to Matlab's sum(A,dim) for array A.

end

>> symperm

```
function A=symperm(A,perm)
```

Permute symmetries according to specified permutation.

Wb, Feb06, 14

>> symrank

```
function [r,nq]=symrank(A)
```

Get symmetry rank for each symmetry.

Wb, Jan09, 17

>> times

```
function C=times(A,B,opt)
```

This overloads the .* operator in Matlab.

Options

--skip-cgc C inherits CGC space of tensor A *as it is*
(yet only for the symmetry sectors shared with B).

Wb, Jul04, 12

>> trace

function x=trace(A,I [,opts])

trace of QSpace operator

Usage 1: A=trace(A) - regular trace of rank-2 object

Usage 2: A=trace(A,I)

generalized trace with pairwise specification of indices
to contract I = [i1 i2; j1 j2; ...]

Wb,Sep11,06 ; Wb,Apr10,15

>> transpose

function A=transpose(A)

This function overloads the .' operator in Matlab
for 'element wise transpose'.

>> uminus

function A=uminus(A)

overloading the unary minus operator (as in -A)

Wb,Sep17,06

>> unmark

Usage #1: function A=unmark(A)

Remove all marks from itags, where marks are considered
trailing special characters other than conjugate flag '*'
such as [~~'+-].

Usage #2: function unmark(A1,A2,...)

With no return argument requested, all modified A's
are assigned back to caller by their inputname
(hence, in this case input arguments cannot be r-values
out of expressions, but must be actual variables).

See also QSpace/

Wb,Jun29,23

>> untag

Usage #1: function A=untag(A)

Remove itags from specified QSpace A
i.e., all characters other than trailing conjugate flag `\*`.

Usage #2: function untag(A1,A2,...)

With no return argument requested, all modified A's
are assigned back to caller by their inputname
(hence, in this case input arguments cannot be r-values
out of expressions, but must be actual variables).

See also QSpace/
Wb,May11,17

>> uplus

function A=uplus(A)

This overloads the unary plus operator (as in +A).

Wb,Sep17,06